React + Vite Prototype Plan: Gemma AI Interface • Dark Mistral Styling

Complete blueprint — stack, design system, file structure, code skeleton, and setup steps.

🎯 Core Concept

- Purpose: Clean, high-performance AI chat interface for interacting with Google Gemma models
- Vibe: Mistral-inspired dark theme — deep charcoal backgrounds, sharp contrast, neon accents, minimal clutter, terminal/IDE aesthetic
- Tech: React 18 + Vite 5 + TypeScript + Tailwind CSS + Lucide Icons
- Status: Frontend prototype (UI + mock API calls; connect to Gemma API later)

🎨 Design System: Dark Mistral Style

Color Palette

Role	Hex Code
Background (main)	#0F1117
Background (surface)	#1A1C23
Background (hover)	#242730
Border / Divider	#2F323D
Text primary	#E6E6E6
Text secondary	#949BA5
Accent primary	#7C3AED (violet)
Accent secondary	#06B6D4 (cyan)
User message bg	#1E293B
Gemma message bg	#1F2229

Typography

- Sans: Inter / system-ui
- Mono: JetBrains Mono / Consolas
- Clean hierarchy: no excessive sizing

Key Style Traits

- Sharp corners (4px–6px radius, no rounded bubbles)
- Subtle glow on active inputs
- Thin borders, soft shadows
- Smooth fade-in for messages
- Loading indicator: pulsing accent dots

📁 Project Structure

plaintext
gemma-mistral-proto/
├── public/
├── src/
│   ├── components/
│   │   ├── ChatMessage.tsx
│   │   ├── ChatInput.tsx
│   │   ├── Header.tsx
│   │   └── TypingIndicator.tsx
│   ├── hooks/
│   │   └── useGemmaChat.ts
│   ├── types/
│   │   └── chat.ts
│   ├── utils/
│   │   └── gemmaApi.ts
│   ├── App.tsx
│   ├── main.tsx
│   ├── index.css
│   └── vite-env.d.ts
├── .gitignore
├── index.html
├── package.json
├── tsconfig.json
├── vite.config.ts
└── tailwind.config.js

🚀 Step 1: Setup Project

bash
Create Vite + React + TS project
npm create vite@latest gemma-mistral-proto -- --template react-ts

cd gemma-mistral-proto

Install dependencies
npm install tailwindcss postcss autoprefixer lucide-react
npm install -D @types/node

Init Tailwind
npx tailwindcss init -p

🚀 Step 2: Configure Files

tailwind.config.js

```
js
/** @type {import('tailwindcss').Config} */
export default {
  content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
  darkMode: "class",
  theme: {
    extend: {
      colors: {
        mistral: {
          bg: "#0F1117",
          surface: "#1A1C23",
          hover: "#242730",
          border: "#2F323D",
          user: "#1E293B",
          gemma: "#1F2229",
        },
        accent: {
          purple: "#7C3AED",
          cyan: "#06B6D4",
        },
      },
      fontFamily: {
        sans: ["Inter", "system-ui", "sans-serif"],
        mono: ["JetBrains Mono", "monospace"],
      },
    },
  },
  plugins: [],
};
```

src/index.css

```
css
@tailwind base;
@tailwind components;
@tailwind utilities;

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
```

```
}

html,
body,
#root {
  height: 100%;
  background-color: #0f1117;
  color: #e6e6e6;
  font-family: "Inter", system-ui, sans-serif;
}

/* Custom scrollbar */
::-webkit-scrollbar {
  width: 4px;
}
::-webkit-scrollbar-thumb {
  background: #2f323d;
  border-radius: 2px;
}
```

🚀 Step 3: Core Code Skeleton

src/types/chat.ts

```
typescript
export interface Message {
  id: string;
  role: "user" | "assistant";
  content: string;
  timestamp: Date;
}

export interface ChatState {
  messages: Message[];
  isLoading: boolean;
  error: string | null;
}
```

src/hooks/useGemmaChat.ts

```
typescript
import { useState, useCallback } from "react";
import type { Message } from "../types/chat";
```

```
export function useGemmaChat() {
  const [messages, setMessages] = useState<Message[]>([]);
  const [isLoading, setIsLoading] = useState(false);

  const sendMessage = useCallback(async (content: string) => {
    if (!content.trim()) return;

    const userMsg: Message = {
      id: Date.now().toString(),
      role: "user",
      content,
      timestamp: new Date(),
    };

    setMessages((prev) => [...prev, userMsg]);
    setIsLoading(true);

    // Replace with real Gemma API call
    setTimeout(() => {
      const reply: Message = {
        id: (Date.now() + 1).toString(),
        role: "assistant",
        content: `Gemma: Received your message — "${content}". This is a prototype
response.`,
        timestamp: new Date(),
      };
      setMessages((prev) => [...prev, reply]);
      setIsLoading(false);
    }, 1200);
  }, []);

  return { messages, isLoading, sendMessage };
}
```

src/App.tsx

```
tsx
import { useState, type FormEvent } from "react";
import { useGemmaChat } from "./hooks/useGemmaChat";
import { Header } from "./components/Header";
import { ChatMessage } from "./components/ChatMessage";
import { ChatInput } from "./components/ChatInput";
import { TypingIndicator } from "./components/TypingIndicator";

function App() {
  const { messages, isLoading, sendMessage } = useGemmaChat();
  const [input, setInput] = useState("");
```

```
  const handleSubmit = (e: FormEvent) => {
    e.preventDefault();
    sendMessage(input);
    setInput("");
  };

  return (
    <div className="flex flex-col h-full bg-mistral-bg text-gray-100">
      <Header />

      <main className="flex-1 overflow-y-auto px-4 py-6 max-w-4xl mx-auto w-full">
        {messages.length === 0 && (
          <div className="flex flex-col items-center justify-center h-full text-center space-y-4
py-16">
            <h2 className="text-2xl font-semibold">Gemma AI Assistant</h2>
            <p className="text-gray-400 max-w-md">
              Dark Mistral interface prototype. Enter a prompt below to start.
            </p>
          </div>
        )}

        <div className="space-y-4">
          {messages.map((msg) => (
            <ChatMessage key={msg.id} message={msg} />
          ))}
          {isLoading && <TypingIndicator />}
        </div>
      </main>

      <footer className="border-t border-mistral-border p-4 max-w-4xl mx-auto w-full">
        <ChatInput
          value={input}
          onChange={setInput}
          onSubmit={handleSubmit}
          disabled={isLoading}
        />
      </footer>
    </div>
  );
}

export default App;
```

src/components/ChatMessage.tsx

tsx

```
import type { Message } from "../types/chat";
import { User, Sparkles } from "lucide-react";

interface Props {
  message: Message;
}

export function ChatMessage({ message }: Props) {
  const isUser = message.role === "user";

  return (
    <div className={`flex gap-3 ${isUser ? "flex-row-reverse" : ""}`}>
      <div
        className={`flex items-center justify-center w-8 h-8 rounded
          ${isUser ? "bg-accent-purple/20 text-accent-purple" : "bg-accent-cyan/20
text-accent-cyan"}`}
      >
        {isUser ? <User size={16} /> : <Sparkles size={16} />}
      </div>

      <div
        className={`max-w-[80%] px-4 py-3 rounded-md leading-relaxed
          ${isUser ? "bg-mistral-user" : "bg-mistral-gemma border border-mistral-border"}`}
      >
        <p className="whitespace-pre-wrap">{message.content}</p>
        <span className="text-xs text-gray-500 mt-1 block">
          {message.timestamp.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit" })}
        </span>
      </div>
    </div>
  );
}
```

src/components/ChatInput.tsx

tsx

```
import type { FormEvent, ChangeEvent } from "react";
import { Send } from "lucide-react";

interface Props {
  value: string;
  onChange: (val: string) => void;
  onSubmit: (e: FormEvent) => void;
  disabled: boolean;
}

export function ChatInput({ value, onChange, onSubmit, disabled }: Props) {
```

```
  return (
    <form onSubmit={onSubmit} className="relative">
      <input
        type="text"
        value={value}
        onChange={(e: ChangeEvent<HTMLInputElement>) => onChange(e.target.value)}
        placeholder="Message Gemma…"
        disabled={disabled}
        className="w-full bg-mistral-surface border border-mistral-border rounded-md
          px-4 py-3 pr-12 text-gray-100 focus:outline-none focus:border-accent-purple
          disabled:opacity-50 transition-colors"
      />
      <button
        type="submit"
        disabled={disabled || !value.trim()}
        className="absolute right-3 top-1/2 -translate-y-1/2 text-gray-400
          hover:text-accent-purple disabled:opacity-30 transition-colors"
      >
        <Send size={18} />
      </button>
    </form>
  );
}
```

src/components/Header.tsx

```
tsx
export function Header() {
  return (
    <header className="border-b border-mistral-border px-6 py-3 flex items-center
justify-between">
      <h1 className="text-lg font-semibold flex items-center gap-2">
        <span className="w-2 h-2 rounded-full bg-accent-cyan animate-pulse"></span>
        Gemma AI
      </h1>
      <span className="text-sm text-gray-500">Dark Mistral Style</span>
    </header>
  );
}
```

src/components/TypingIndicator.tsx

```
tsx
export function TypingIndicator() {
  return (
    <div className="flex gap-3 items-center text-gray-400">
```

```
      <div className="w-8 h-8 rounded bg-accent-cyan/20 flex items-center justify-center">
        <span className="text-accent-cyan text-sm">✨</span>
      </div>
      <div className="flex gap-1">
        {[0, 1, 2].map((i) => (
          <span
            key={i}
            className="w-2 h-2 rounded-full bg-accent-cyan"
            style={{
              animation: `bounce 1.4s infinite`,
              animationDelay: `${i * 0.2}s`,
            }}
          />
        ))}
      </div>
      <style>{`
        @keyframes bounce {
          0%, 80%, 100% { transform: translateY(0); }
          40% { transform: translateY(-4px); }
        }
      `}</style>
    </div>
  );
}
```

🧪 Run & Test

bash
npm run dev

- Visit http://localhost:5173
- Type a message → see simulated Gemma response
- Check mobile responsiveness

🔌 Next Steps — Connect Real Gemma

1. Get API access: Google AI Studio / Hugging Face / local inference
2. Update src/utils/gemmaApi.ts with fetch/axios call
3. Replace the setTimeout mock in useGemmaChat.ts with real API request
4. Add loading states, error handling, and token streaming

✅ Deliverables Checklist

Vite + React + TS project initialized
Tailwind configured with Dark Mistral palette
All components created & styled
Chat logic working with mock responses
Responsive layout tested
API integration skeleton ready

Would you like me to turn this into a single downloadable ZIP-ready project file, or add streaming response / markdown rendering next?

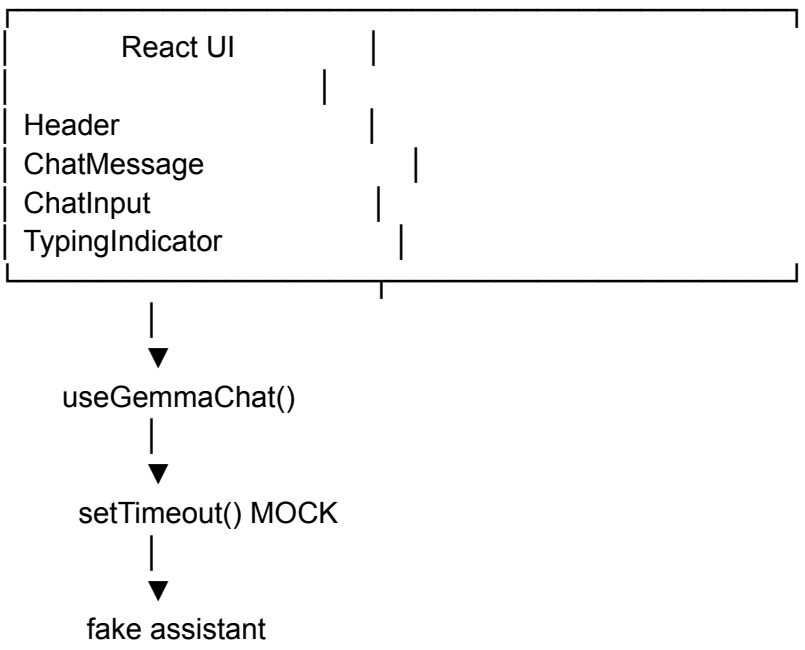
Yes. The prototype is a good UI shell, but technically it is still several layers away from being a production-grade AI client. The biggest gap is that the current architecture treats “Gemma chat” as a simple sendMessage → string response operation. A real application needs to model conversations, streaming, cancellation, persistence, API/security boundaries, multimodal input, errors, configuration, observability, and eventually tools/agents.

One important update: Google's current API documentation now recommends the Interactions API as the primary primitive for newer Gemini/Gemma application workflows, while generateContent and streamGenerateContent remain available. The current SDK is @google/genai.

Complete Technical Gap Analysis

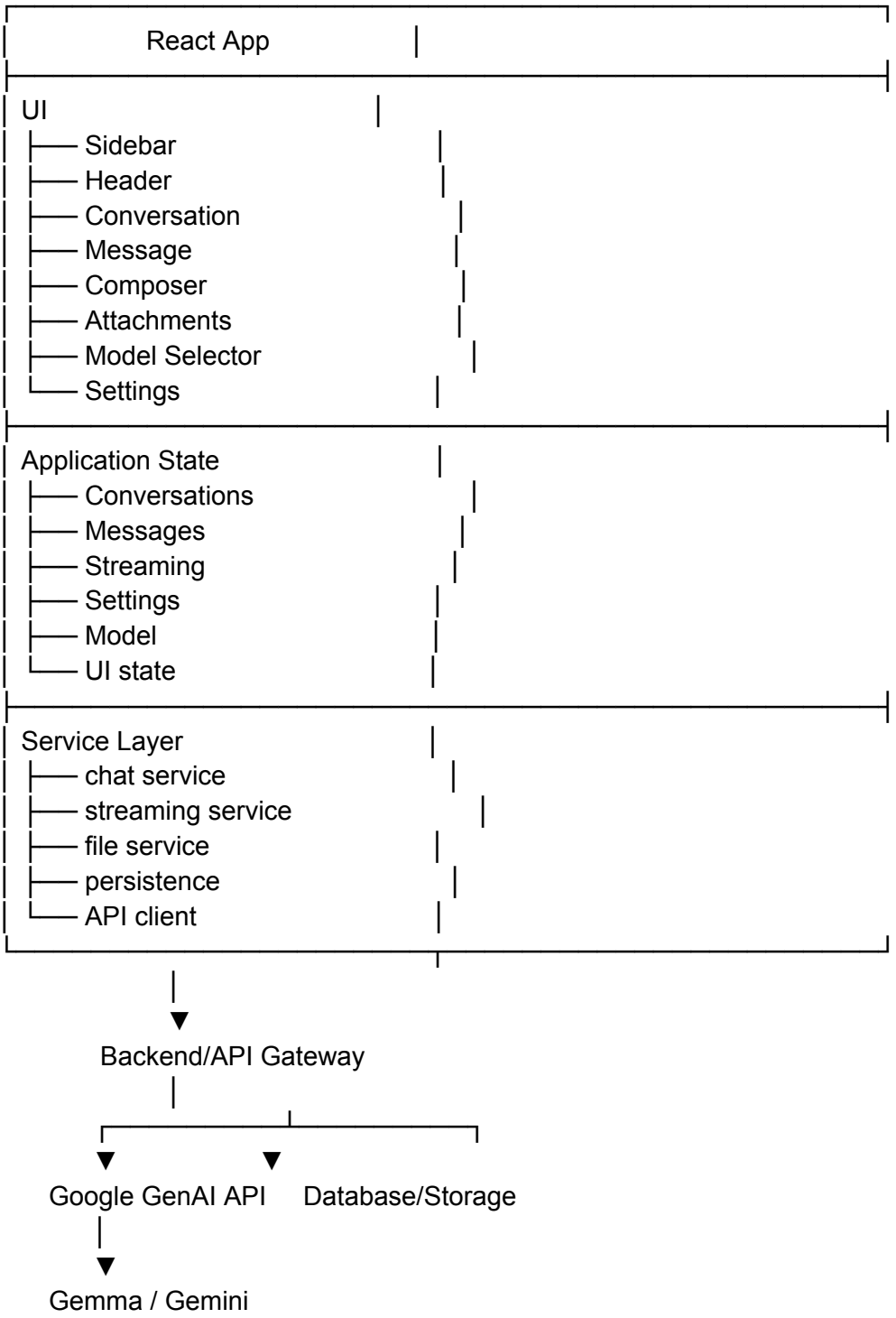
1. Current architecture

The existing prototype effectively has this architecture:



That is perfectly adequate for proving the visual concept.

A production architecture should become:



The critical architectural change is:

> Do not make the React browser the owner of your production API credentials.

For a real deployment, use a backend or server-side API layer.

---

2. Missing project infrastructure

The current file tree is too small for a serious application.

I would expand it to something closer to:

```
gemma-mistral/
│
├── public/
│   ├── favicon.svg
│   └── logo.svg
│
├── src/
│   │
│   ├── app/
│   │   ├── App.tsx
│   │   ├── routes.tsx
│   │   └── providers.tsx
│   │
│   ├── components/
│   │   ├── chat/
│   │   │   ├── ChatMessage.tsx
│   │   │   ├── MessageActions.tsx
│   │   │   ├── MessageContent.tsx
│   │   │   ├── MessageMetadata.tsx
│   │   │   ├── StreamingMessage.tsx
│   │   │   └── TypingIndicator.tsx
│   │   │
│   │   ├── composer/
│   │   │   ├── ChatInput.tsx
│   │   │   ├── AttachmentButton.tsx
│   │   │   ├── AttachmentPreview.tsx
│   │   │   ├── SendButton.tsx
│   │   │   └── StopButton.tsx
│   │   │
│   │   ├── layout/
│   │   │   ├── Header.tsx
│   │   │   ├── Sidebar.tsx
│   │   │   ├── MainLayout.tsx
│   │   │   └── MobileNav.tsx
│   │   │
│   │   ├── model/
```

```
│   │   │   ├── ModelSelector.tsx
│   │   │   ├── ModelBadge.tsx
│   │   │   └── ModelInfo.tsx
│   │   │
│   │   ├── settings/
│   │   │   ├── SettingsPanel.tsx
│   │   │   ├── GenerationSettings.tsx
│   │   │   └── AppearanceSettings.tsx
│   │   │
│   │   └── ui/
│   │       ├── Button.tsx
│   │       ├── Modal.tsx
│   │       ├── Dropdown.tsx
│   │       ├── Tooltip.tsx
│   │       └── Toast.tsx
│   │
│   ├── hooks/
│   │   ├── useChat.ts
│   │   ├── useStreaming.ts
│   │   ├── useConversations.ts
│   │   ├── useAutoScroll.ts
│   │   ├── useKeyboardShortcuts.ts
│   │   └── useLocalStorage.ts
│   │
│   ├── services/
│   │   ├── api/
│   │   │   ├── client.ts
│   │   │   ├── chat.ts
│   │   │   ├── models.ts
│   │   │   └── files.ts
│   │   │
│   │   ├── storage/
│   │   │   ├── conversations.ts
│   │   │   └── settings.ts
│   │   │
│   │   └── markdown/
│   │       └── renderer.ts
│   │
│   ├── stores/
│   │   ├── chatStore.ts
│   │   ├── conversationStore.ts
│   │   └── settingsStore.ts
│   │
│   ├── types/
│   │   ├── chat.ts
│   │   ├── api.ts
│   │   ├── model.ts
│   │   ├── attachment.ts
```

```
│   │   └── settings.ts
│   │
│   ├── utils/
│   │   ├── ids.ts
│   │   ├── formatting.ts
│   │   ├── validation.ts
│   │   └── errors.ts
│   │
│   ├── App.tsx
│   ├── main.tsx
│   └── index.css
│
├── server/
│   ├── index.ts
│   ├── routes/
│   │   ├── chat.ts
│   │   ├── models.ts
│   │   └── files.ts
│   ├── services/
│   │   └── gemma.ts
│   └── middleware/
│       ├── auth.ts
│       ├── rateLimit.ts
│       └── errors.ts
│
├── tests/
│   ├── components/
│   ├── hooks/
│   └── services/
│
├── .env.example
├── .gitignore
├── package.json
├── tsconfig.json
├── vite.config.ts
├── tailwind.config.js
└── README.md
```

That gives you room to grow without turning App.tsx into a 1,500-line kitchen sink.

---

3. The current message model is insufficient

Current:

```
interface Message {
```

```
  id: string;
  role: "user" | "assistant";
  content: string;
  timestamp: Date;
}
```

A real message needs considerably more information.

For example:

```
export type MessageRole =
  | "system"
  | "user"
  | "assistant"
  | "tool";

export interface Message {
  id: string;
  conversationId: string;
  role: MessageRole;

  content: MessageContent[];

  createdAt: string;
  updatedAt?: string;

  status:
    | "pending"
    | "streaming"
    | "complete"
    | "error"
    | "cancelled";

  model?: string;

  usage?: {
    inputTokens?: number;
    outputTokens?: number;
    totalTokens?: number;
  };

  attachments?: Attachment[];

  error?: {
    code: string;
    message: string;
  };
}
```

And:

```
export interface MessageContent {
  type: "text" | "image" | "file" | "audio" | "video";
  text?: string;
  url?: string;
  mimeType?: string;
}
```

This matters because the current content: string model prevents the UI from naturally supporting multimodal messages.

---

4. Conversation management is completely missing

The prototype currently has one conversation that disappears when the page refreshes.

A serious interface needs:

```
Conversations
│
├── New Chat
│
├── Today
│   ├── Explain quantum computing
│   ├── Build React component
│   └── Research Australian history
│
├── Yesterday
│   ├── ...
│
└── Older
    └── ...
```

Required capabilities:

New conversation

Rename conversation

Delete conversation

Duplicate conversation

Search conversations

Pin conversation

Archive conversation

Conversation timestamps

Automatic title generation

Persistent history

Active conversation selection

Empty state

Conversation recovery after refresh

A basic model:

```
interface Conversation {
  id: string;
  title: string;
  model: string;
  createdAt: string;
  updatedAt: string;
  messages: Message[];
}
```

For a prototype, localStorage is sufficient.

For production:

```
React
  ↓
API
  ↓
Database
```

---

5. Streaming is a major missing feature

The current interface waits 1.2 seconds and then inserts a complete response.

That makes it feel like a mockup rather than an AI application.

Google's current API supports streamed generation, and the newer Interactions API uses SSE streaming events.

The desired experience is:

Gemma is responding...

The answer begins appearing
character/token by token...

Conceptually:

```
const assistantMessage = createEmptyMessage();

for await (const chunk of stream) {
  appendToMessage(
    assistantMessage.id,
    chunk.text
  );
}
```

The UI therefore needs:

```
pending
  ↓
streaming
  ↓
complete
```

rather than simply:

```
loading
  ↓
done
```

---

6. Stop generation is missing

Once streaming exists, the user needs:

[Stop]

The implementation should use AbortController.

```
const controller = new AbortController();

fetch(url, {
  signal: controller.signal
});
```

Then:

```
controller.abort();
```

The message should become:

```
status: "cancelled"
```

rather than simply disappearing.

---

7. Error handling is almost entirely missing

Currently there is no meaningful error path.

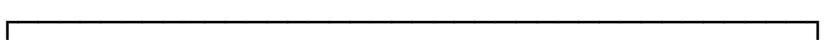
You need to distinguish:

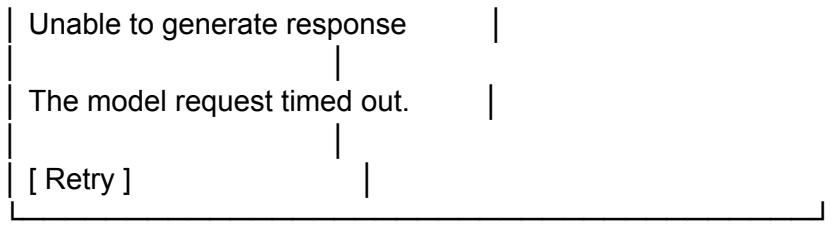
Network error
API error
Authentication error
Rate limit
Invalid request
Model unavailable
Context too large
File upload failure
Streaming interruption
Unknown server error

For example:

```
interface ApiError {
  code: string;
  message: string;
  retryable: boolean;
  status?: number;
}
```

UI:





A failed assistant message should also have a retry action.

---

8. Markdown rendering is missing

The current:

{message.content}

will make AI responses look primitive.

AI output commonly contains:

headings

bold

italics

lists

tables

links

blockquotes

inline code

fenced code

mathematical notation

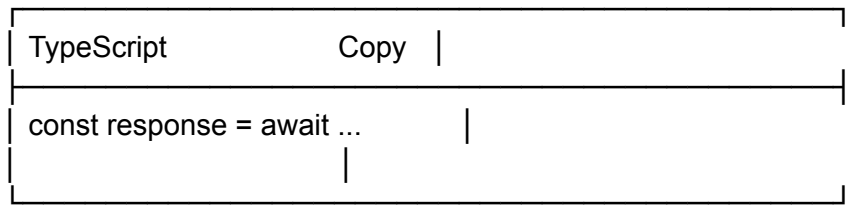
You therefore need a markdown rendering pipeline.

Conceptually:

Gemma output

↓
Markdown parser
↓
Sanitization
↓
React renderer
↓
Styled HTML

And code blocks need:



The Copy button should copy only the code, not the surrounding UI.

---

9. Syntax highlighting is missing

For a developer-oriented Mistral/IDE aesthetic, code presentation is essential.

Support at minimum:

JavaScript
TypeScript
Python
JSON
HTML
CSS
Bash
SQL
Markdown
YAML

The renderer should detect:

```typescript
...
```

and apply the appropriate language.

---

10. Message actions are missing

Every assistant message should eventually support:

Copy
Regenerate
Edit
Continue
Like
Dislike
Share

Potential UI:

Gemma response

[Copy] [Regenerate] [···]

For user messages:

[Edit]

Editing should ideally:

User message
↓
Edit
↓
Change prompt
↓
Resubmit
↓
Create new response branch

---

11. Regeneration requires conversation branching

This is subtle but important.

If the user regenerates:

User
└── Assistant A

Regenerate

User
├── Assistant A
└── Assistant B

You should not simply overwrite the original answer.

A mature conversation model supports:

parentMessageId
branchId

This enables:

‹ 1 / 3 ›

between alternative responses.

---

12. Auto-scroll behaviour is missing

A chat interface needs intelligent scrolling.

There are two different states:

User is at bottom

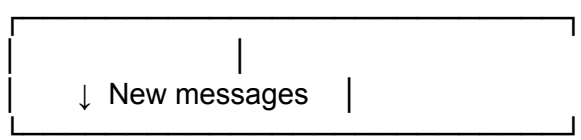
Automatically scroll while streaming.

User has scrolled upward

Do not yank them back down.

Instead:

↓ New messages



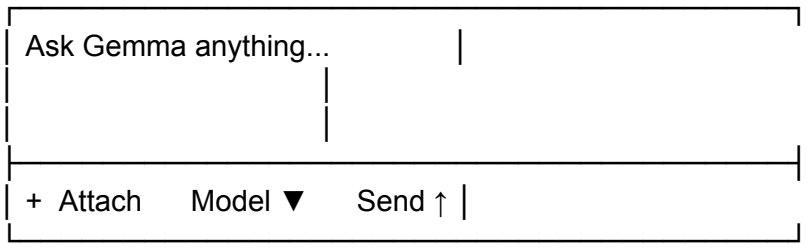
This is one of those tiny details that separates “web page containing chat” from “actual chat application.”

---

13. Input needs to become a composer

The current single-line <input> is too restrictive.

Use:



Features:

multiline input

Enter = send

Shift+Enter = newline

textarea auto-grow

character/token indication

attachment button

model selector

stop button while streaming

disabled state

drag/drop

paste image

keyboard shortcuts

---

14. File and image attachments are missing

The API architecture should allow multimodal content.

Google's generation API supports text, images, audio, video and PDF inputs.

The frontend therefore needs:

```
interface Attachment {
  id: string;
  name: string;
  mimeType: string;
  size: number;

  status:
    | "uploading"
    | "processing"
    | "ready"
    | "error";

  url?: string;
}
```

UI:

```
┌──────────────────────────────────────────┐
│ 📎 document.pdf                          │
│ 2.4 MB                  ×                │
└──────────────────────────────────────────┘
```

For images:

```
┌──────────────────┐
│                  │
│    image         │
│                  │
└──────────────────┘
```

---

15. Drag and drop is missing

Support:

Drag files here

with:

dragenter

dragover

dragleave

drop

MIME validation

file-size validation

upload progress

error state

---

16. Model selection is missing

The application should not hard-code one model.

Add:

Model

```
┌──────────────────────────────────┐
│ Gemma 4 31B IT        ✓ │
│ Gemma 4 26B MoE IT      │
│ Gemini ...              │
└──────────────────────────────────┘
```

The current Google documentation lists Gemma models available through the Interactions API, including Gemma 4 variants.

However, the actual available model list should come from configuration or the API rather than being permanently embedded in the UI.

---

17. Generation parameters are missing

Advanced settings should expose appropriate parameters such as:

Temperature
Top P

Top K
Maximum output tokens
System instruction

Potential UI:

Generation

Temperature 0.7
──────────●──────────

Top P 0.95
──────────●──────────

Max output 4096

Not every model necessarily supports every parameter, so the frontend should derive available controls from model capabilities.

---

18. System instructions are missing

A serious AI interface needs a system-prompt layer.

For example:

System instructions

You are a concise technical assistant...

Internally:

system
↓
conversation history
↓
user message

The system instruction should not be treated as an ordinary user message.

---

19. Token/context management is missing

This becomes critical for long conversations.

The application needs to understand:

Conversation
↓
Token count
↓
Context budget

Eventually:

Messages 1–30
↓
Context too large
↓
Summarise older messages
↓
Keep recent messages
↓
Send compact context

Useful UI:

Context
 72%

12,480 / 16,384 tokens

The exact limits should come from the selected model's capabilities rather than being hard-coded.

---

20. Conversation summarisation is missing

For long-lived chats:

Old messages
↓
Summarisation
↓
Conversation memory
↓
Recent messages

Possible internal structure:

```
interface ConversationContext {
  summary?: string;
  recentMessageIds: string[];
}
```

This dramatically improves scalability.

---

21. Persistent storage is missing

At the moment:

```
Refresh browser
      ↓
Everything gone
```

Minimum prototype:

```
localStorage
```

Better browser architecture:

```
IndexedDB
```

Production:

```
PostgreSQL
    +
object storage
```

A hybrid approach is particularly attractive:

```
Local UI cache
      +
server persistence
```

---

22. Authentication is missing

For a real application:

```
Login
 ↓
Session
```

↓
User
↓
Conversations

Without authentication, you cannot reliably associate:

user → conversations → files → settings

Possible architecture:

OAuth / Auth provider
↓
Backend session
↓
API authorization

---

23. API key security is missing

This is one of the biggest technical issues in the current plan.

Do not ship a private production API key inside:

VITE_GEMINI_API_KEY

because Vite environment variables prefixed with VITE_ are exposed to the client bundle.

Instead:

Browser
↓
/api/chat
↓
Backend
↓
GEMINI_API_KEY
↓
Google

Environment:

GEMINI_API_KEY=...

only on the server.

---

24. Backend API is missing

A minimal backend might expose:

POST /api/chat
POST /api/chat/stream

GET /api/models

GET /api/conversations
POST /api/conversations

GET /api/conversations/:id
DELETE /api/conversations/:id

POST /api/files
DELETE /api/files/:id

The backend becomes the security and orchestration boundary.

---

25. Streaming transport needs to be designed

For chat streaming:

Browser
  │
  │ POST /api/chat/stream
  ▼
Backend
  │
  │ stream
  ▼
Google API

Then:

Google
↓
backend
↓
SSE
↓

React

SSE is a natural fit for text generation because Google's streaming APIs already expose incremental events.

---

26. Tool/function calling is missing

This is a major future capability.

Modern Gemini APIs support tools including Google Search and custom function calling.

Architecture:

User
↓
Model
↓
Tool call
↓
Application executes function
↓
Tool result
↓
Model
↓
Final answer

Example:

User:
What's the weather?

Gemma:
CALL getWeather({
  location: "Brisbane"
})

Application:
32°C

Gemma:
It's currently 32°C...

The UI should eventually display tool activity:

⚙ Calling weather service...
✓ Tool completed

---

27. Search / grounding layer is missing

If the application will answer current-information questions, it needs a grounding/search strategy.

Potential architecture:

Prompt
↓
Model decides whether external information needed
↓
Search/tool
↓
Results
↓
Model
↓
Answer + sources

The Gemini API currently documents managed tools such as Google Search, as well as custom functions.

---

28. Structured output is missing

For applications that need machine-readable results:

Model
↓
JSON schema
↓
Validated object
↓
React component

For example:

```
{
  "title": "Project Plan",
  "priority": "high",
```

```
  "tasks": []
}
```

Google's current API supports structured outputs and streaming structured responses.

This opens the door to much more interesting UI than simply rendering text.

---

29. Tool execution UI is missing

If agents/tools are added, messages should no longer be just:

content: string

You may need:

```
type MessagePart =
  | TextPart
  | ToolCallPart
  | ToolResultPart
  | FilePart
  | ImagePart
  | CodePart;
```

Then the UI can render:

```
┌──────────────────────────────────────────────┐
│ 🔧 get_weather                               │
│                                              │
│ location: Brisbane                           │
│                                              │
│ ✓ completed                                  │
└──────────────────────────────────────────────┘
```

---

30. Markdown security is missing

Rendering AI-generated HTML introduces security considerations.

You need:

```
Markdown
↓
Parser
```

↓
Sanitizer
↓
Safe React rendering

Never blindly inject model output using:

dangerouslySetInnerHTML

without sanitization.

---

31. Accessibility is missing

The prototype should add:

semantic landmarks

labels

keyboard navigation

visible focus states

ARIA labels where needed

screen-reader announcements

reduced-motion support

sufficient contrast

accessible error messages

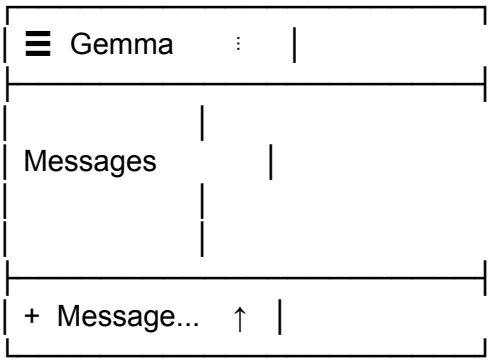
Streaming messages should not spam screen readers with every token.

---

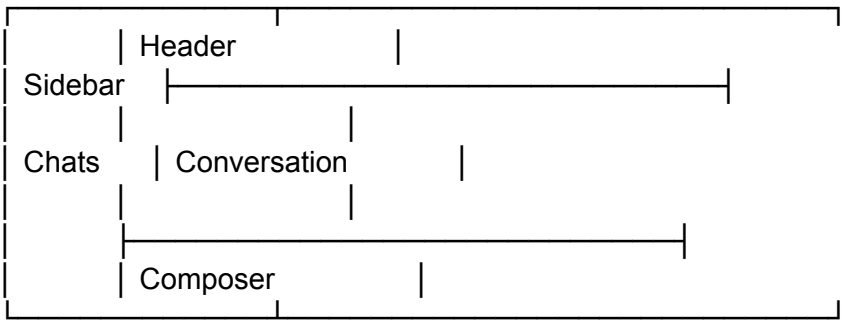
32. Mobile architecture needs improvement

The current responsive layout is basic.

A real mobile UI should become:



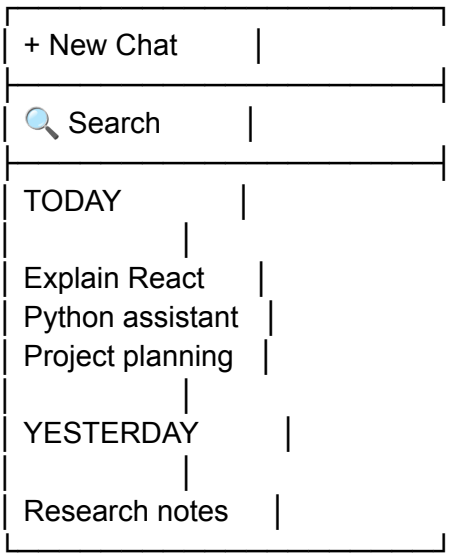
Desktop:



---

33. Sidebar is missing

This should probably be one of the next UI components.



---

34. Settings system is missing

At minimum:

Appearance

Theme
○ Dark
○ System
○ Light

Chat

Enter sends message ON
Show timestamps ON
Compact messages OFF

Generation

Temperature
Max output
System prompt

Privacy

Save conversations ON
Save uploaded files OFF

---

35. Theme system should use CSS variables

Instead of scattering hex values through Tailwind:

```
:root {
  --bg: #0f1117;
  --surface: #1a1c23;
  --hover: #242730;
  --border: #2f323d;

  --text-primary: #e6e6e6;
  --text-secondary: #949ba5;

  --accent-primary: #7c3aed;
  --accent-secondary: #06b6d4;
}
```

This makes future themes much easier.

You could eventually support:

Mistral Dark
Gemma Dark
High Contrast
OLED
Light
Custom

---

36. Animation system is incomplete

The current typing animation works, but production UI needs a coherent motion system.

Examples:

Message enter
fade + translateY

Sidebar
slide

Modal
fade + scale

Streaming cursor
pulse

Button
micro interaction

Respect:

```
@media (prefers-reduced-motion: reduce) {
  ...
}
```

---

37. Loading states need to be granular

Instead of one:

isLoading

use distinct states:

```
status:
  | "idle"
  | "submitting"
  | "streaming"
  | "uploading"
  | "processing"
  | "error";
```

Otherwise the interface cannot tell whether:

Gemma is generating

or:

file is uploading

or:

server is reconnecting

---

38. Network resilience is missing

Real networks fail.

You need:

```
Request
 ↓
Timeout?
 ↓
Retry?
 ↓
Reconnect?
```

For streaming:

```
stream interrupted
      ↓
preserve partial response
      ↓
```

show error
↓
[Continue] [Retry]

Do not discard 1,800 tokens because token 1,801 encountered a network failure.

---

39. Rate limiting and abuse protection

Backend requirements:

Authentication
↓
Rate limiter
↓
Request validation
↓
Model API

Possible controls:

requests/minute
tokens/minute
maximum message size
maximum file size
maximum files/request

---

40. Input validation is missing

Validate:

empty messages
maximum message length
unsupported files
oversized files
malformed API parameters
invalid model IDs

Both:

client

and:

server

must validate.

Client validation is UX.

Server validation is security.

---

41. Observability is missing

For production debugging:

Request ID
Conversation ID
Message ID
Model
Latency
Input tokens
Output tokens
Error code

Example:

requestId: req_91a...
model: gemma...
latency: 2.84s
inputTokens: 482
outputTokens: 931
status: success

Do not log sensitive conversation content by default.

---

42. Analytics should be privacy-conscious

Useful metrics:

messages sent
response latency
stream completion rate
error rate
model usage

token usage

Avoid collecting conversation contents unless there is a clearly justified requirement and appropriate user controls.

---

43. Testing infrastructure is missing

Add:

Vitest
React Testing Library
Playwright

Test:

Unit

message formatting
validation
API transformations
token calculations
storage

Component

ChatInput
ChatMessage
Sidebar
ModelSelector

Integration

send message
stream response
retry
cancel
upload file

E2E

open app
create conversation
send prompt
receive streamed response
refresh

conversation remains

---

44. Type safety needs to extend across the API boundary

Do not have:

frontend type

and separately invented:

backend type

Use shared schemas.

A particularly strong pattern is:

Zod schema
↓
TypeScript type
↓
Request validation
↓
Response validation

This eliminates a large class of frontend/backend mismatch bugs.

---

45. Configuration management is missing

Add:

```
# Server
GEMINI_API_KEY=
DATABASE_URL=

# Optional
AUTH_SECRET=
STORAGE_BUCKET=
```

Frontend:

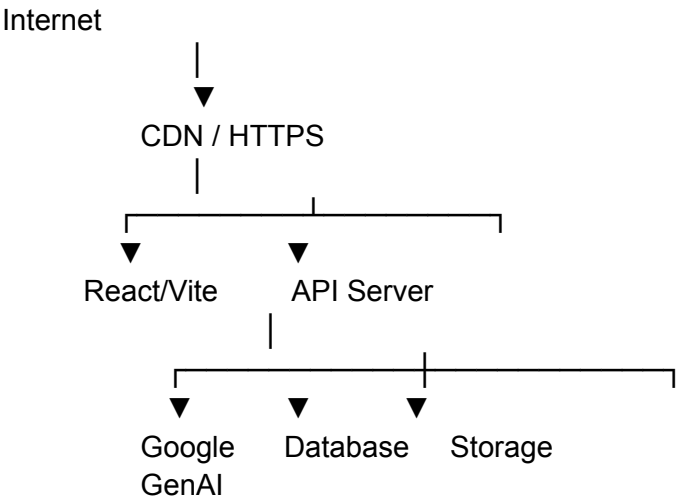
```
VITE_API_BASE_URL=
```

Never put secrets into Vite's public environment variables.

---

46. Deployment architecture is missing

A clean production deployment could be:



Potential deployment platforms include:

Frontend
Vercel / Netlify / Cloudflare

Backend
Cloud Run / Vercel Functions / Node server

Database
PostgreSQL

Files
Object storage

---

47. PWA/offline capability is missing

For a polished client:

Service worker
↓
Cached application shell
↓

Open UI offline

You could preserve:

previous conversations
draft message
settings

and display:

● Offline

rather than a broken page.

---

48. Draft preservation is missing

If the user types 2,000 words and accidentally refreshes:

💥

Avoid this.

Persist draft:

conversationId → draft

using local storage or IndexedDB.

---

49. Keyboard shortcut system is missing

Useful shortcuts:

Enter Send
Shift+Enter New line
Ctrl/Cmd+K Search
Ctrl/Cmd+N New chat
Esc Stop generation
Ctrl/Cmd+/ Shortcuts

A shortcut help modal could show:

Keyboard shortcuts

_____

New chat Ctrl N
Search Ctrl K
Send Enter
New line Shift Enter
Stop Esc

---

50. Search within conversations is missing

Eventually:

Ctrl + K

opens:

Search chats

🔍 quantum computing

2 results

Explain quantum computing
Quantum notes

Search can cover:

conversation titles
message content
metadata

---

51. Export is missing

Useful formats:

Export Markdown
Export JSON
Export TXT
Print / PDF

Example:

Conversation

↓
Export
↓
Markdown

---

52. Import is missing

For portability:

Import conversation

Possible format:

```
{
  "version": 1,
  "conversation": {}
}
```

Version the schema from day one.

---

53. Data deletion/privacy controls are missing

A serious product should provide:

Delete conversation
Delete all conversations
Delete uploaded files
Clear local data
Export my data

And confirmation for destructive operations.

---

54. Content safety/error states need UI

Depending on model/API behaviour, responses may be:

blocked
filtered
empty

incomplete

Do not render these as an ordinary assistant response.

Instead:

⚠ Response unavailable

The model could not provide a response to this request.

---

55. Model capability registry is missing

Rather than:

if (model === "...")

create:

```
interface ModelCapabilities {
  streaming: boolean;
  vision: boolean;
  audio: boolean;
  video: boolean;
  tools: boolean;
  structuredOutput: boolean;
  maxContextTokens?: number;
}
```

Then the UI can dynamically determine:

Model supports image input ✓
Model supports tools ✓
Model supports audio ×

This is much cleaner than hard-coded UI assumptions.

---

56. API abstraction is missing

Do not let components directly know about Google.

Bad:

ChatMessage → Google API

Better:

Chat UI
  ↓
useChat()
  ↓
ChatService
  ↓
API Client
  ↓
Backend
  ↓
Google

This means you can later support:

Gemma
Gemini
OpenAI
Anthropic
Ollama
LM Studio
Hugging Face
local inference

without rebuilding the UI.

---

57. Provider abstraction

A particularly powerful future interface:

```
interface AIProvider {
  sendMessage(
    request: ChatRequest
  ): AsyncIterable<ChatEvent>;
}
```

Then:

GoogleProvider
OllamaProvider
OpenAIProvider
LocalProvider

all implement the same interface.

The UI does not care which model is underneath.

---

58. Local Gemma inference is a separate architecture

If the eventual goal is truly Gemma locally, the architecture changes.

Possible:

React
↓
Local backend
↓
Ollama / llama.cpp / Transformers
↓
Gemma

or:

React
↓
WebGPU/WebAssembly inference
↓
Gemma

The second approach has very different memory, browser compatibility and model-loading considerations.

Therefore the architecture should avoid hard-coding the assumption:

Gemma = Google API

---

59. The current gemmaApi.ts should not be a fetch dump

Instead:

```
export interface ChatRequest {
  conversationId: string;
  messages: Message[];
  model: string;
```

```
  settings: GenerationSettings;
}

export interface ChatStream {
  stream: AsyncIterable<ChatEvent>;
  cancel(): void;
}
```

Then:

```
class ChatService {
  async streamChat(
    request: ChatRequest
  ): Promise<ChatStream> {
    ...
  }
}
```

This keeps API complexity away from React components.

---

60. State management will eventually become necessary

For the current prototype:

useState

is fine.

Once you have:

conversations
messages
streaming
settings
attachments
model
sidebar
search
auth

component-local state becomes unwieldy.

A lightweight store architecture becomes appropriate:

chatStore

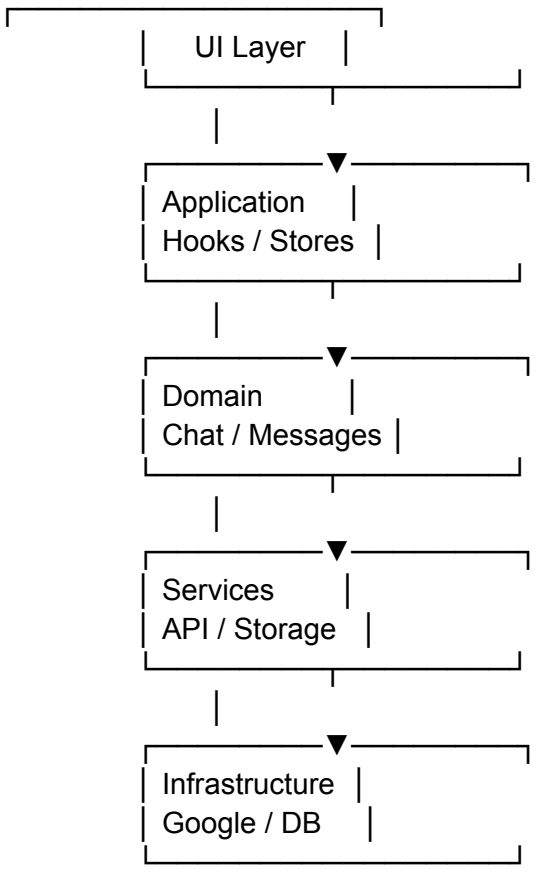
conversationStore
settingsStore

Zustand would be a reasonable option, although it is not strictly required.

---

61. Suggested final frontend architecture

I would structure the actual application around five layers:



That gives you a clean separation between:

what the user sees

and

what the AI infrastructure does.

---

62. Recommended feature roadmap

Rather than implementing everything simultaneously, I would build it in this order.

Phase 1: Turn the prototype into a real chat UI

Add:

multiline composer

proper message model

Markdown

code highlighting

copy buttons

retry

regenerate

stop generation

auto-scroll

error states

sidebar

new conversation

local persistence

At this point it becomes a genuinely usable frontend.

Phase 2: Real Gemma connectivity

Add:

Backend
↓
Google GenAI SDK
↓
Gemma

Use the current Google SDK rather than designing around an obsolete/custom HTTP abstraction. Google's current JavaScript examples use @google/genai.

Implement:

model selection

streaming

cancellation

retries

error translation

token usage

generation parameters

Phase 3: Multimodal

Add:

Images
PDF
Files
Audio
Video

where supported by the selected model/API.

Google's current generation API exposes multimodal request structures for these input types.

Phase 4: Persistence

Add:

Authentication
↓

PostgreSQL
↓
Conversation storage

Plus:

conversation search

titles

rename

archive

delete

export

Phase 5: Agent capabilities

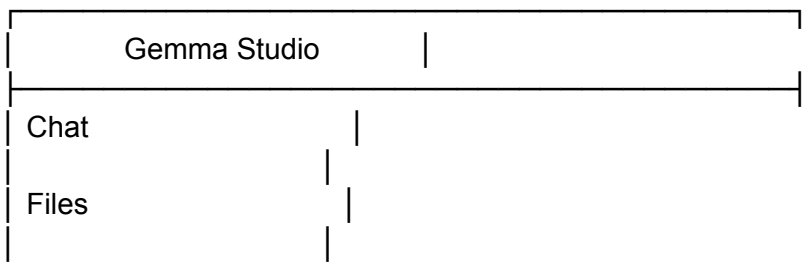
Add:

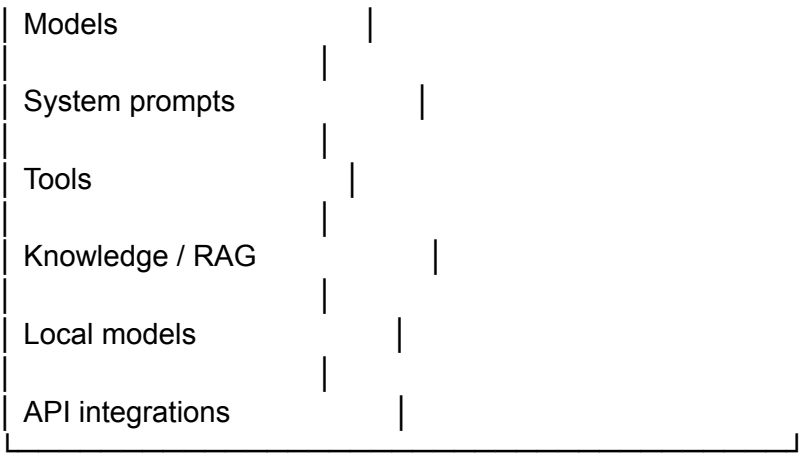
Tools
Search
Function calling
Structured output
Tool result rendering

The current Gemini API explicitly supports built-in tools and custom function calling.

Phase 6: Advanced platform

Eventually:





At that point this stops being merely a “Gemma chat UI” and becomes a model-agnostic AI workstation.

---

63. The most important missing pieces

If we strip away all the optional bells and whistles, these are the critical omissions in the current prototype:

Priority	Missing component	Importance
🔴 P0	Real API/backend	Critical
🔴 P0	API key security	Critical
🔴 P0	Streaming	Critical
🔴 P0	Error handling	Critical
🔴 P0	Abort/cancel	Critical
🔴 P0	Persistent conversation state	Critical
🔴 P0	Proper message schema	Critical
🟠 P1	Markdown renderer	High
🟠 P1	Code highlighting	High
🟠 P1	Conversation sidebar	High
🟠 P1	Model selector	High
🟠 P1	Multiline composer	High
🟠 P1	Attachments	High
🟠 P1	Retry/regenerate	High
🟠 P1	Auto-scroll	High
🟠 P1	Settings	High
🟡 P2	Authentication	Production
🟡 P2	Database	Production
🟡 P2	Search	Production
🟡 P2	Export/import	Useful
🟡 P2	Tool calling	Advanced
🟡 P2	Web/search grounding	Advanced

🟡 P2 Structured output Advanced
🟡 P2 RAG Advanced
🟢 P3 Branching Advanced
🟢 P3 Local inference Advanced
🟢 P3 PWA/offline Optional
🟢 P3 Multi-provider Strategic

---

64. One architectural change I would make immediately

The current plan says:

> src/utils/gemmaApi.ts

I would change that to:

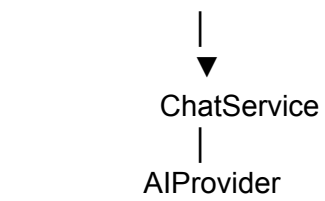
```
src/services/
└── ai/
    ├── AIProvider.ts
    ├── GoogleProvider.ts
    ├── ChatService.ts
    └── types.ts
```

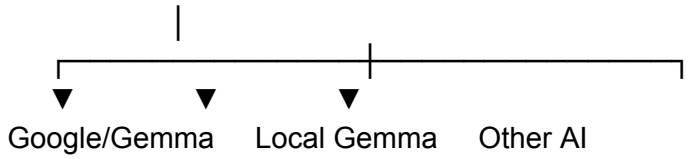
and introduce:

```
interface AIProvider {
  streamChat(
    request: ChatRequest,
    signal?: AbortSignal
  ): AsyncIterable<ChatEvent>;
}
```

That single decision prevents the entire application from becoming tightly coupled to one provider.

Then your eventual architecture becomes:



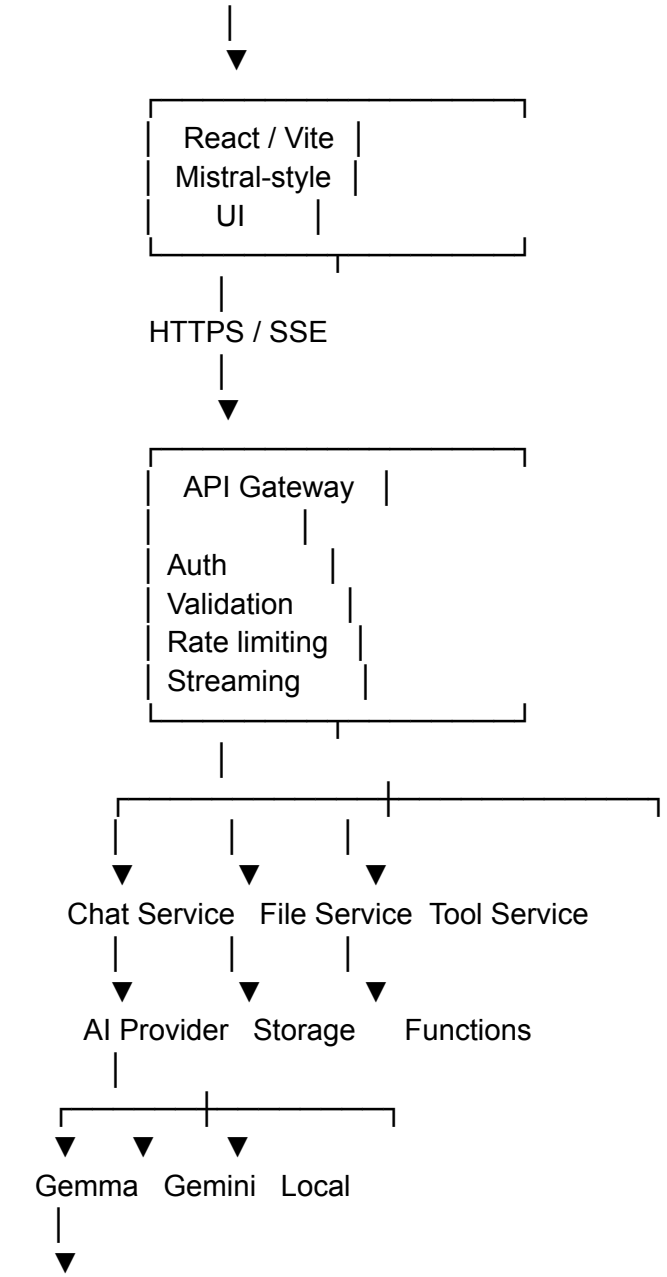


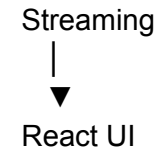
That's the foundation I would build on.

---

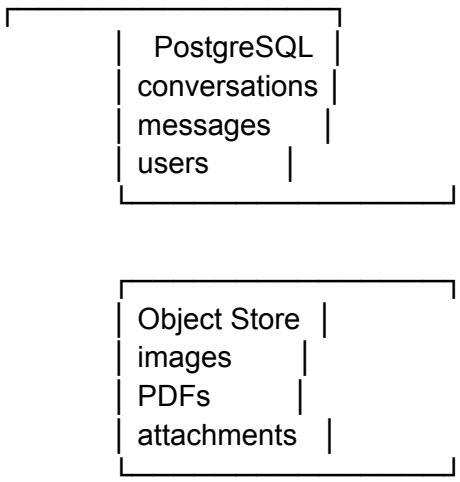
65. Target architecture

The finished system should ultimately look like this:





And alongside it:



Bottom line

The supplied blueprint successfully covers the visual prototype layer, but only roughly 20–30% of the architecture required for a robust AI client.

The immediate next milestone should not simply be "connect gemmaApi.ts."

It should be:

> Prototype → real streaming chat architecture

That means implementing the backend/API boundary, proper message/event types, streaming, cancellation, Markdown/code rendering, persistent conversations, error/retry handling, model selection, and a real composer first.

After that foundation is stable, attachments, tools, search, structured output, RAG, authentication and multi-provider/local inference can be added without having to tear the application apart.

Google's current API direction also makes this a particularly good time to keep the provider layer abstract: the platform now distinguishes standard generation, streaming, real-time Live API, and the newer Interactions API, with Interactions positioned as the recommended primitive for more stateful and agentic applications.

I'll create a comprehensive interactive architecture plan that visualizes your gap analysis as a production-grade system blueprint. Let me first read the visualization skill requirements.

Now let me read the design system reference for proper styling guidance.

Let me also read the skeleton templates for the dashboard structure.

Now I have all the design guidance needed. I'll create a comprehensive interactive architecture dashboard that visualizes your complete production system blueprint — with layered architecture diagrams, priority matrix, phase roadmap, and file structure.

Here's the complete interactive technical plan:

```
html
<html style="margin:0;padding:0;">
<link
href="https://fonts.googleapis.com/css2?family=Sora:wght@400;500;600;700&display=swap
" rel="stylesheet">
<script src="https://cdn.tailwindcss.com"></script>
<script>
(function(){
  var s = document.createElement('style');
  s.textContent =
':root{--bg:#202024;--card:#2a2a2f;--text:#e8e5df;--muted:#9a978f;--accent:#c47046;--accen
t-soft:rgba(196,112,70,.14);--accent2:#7ba8b8;--accent2-soft:rgba(123,168,184,.14);--border
:rgba(232,229,223,.12)}.reveal{opacity:0;transform:translateY(20px);animation:fadeUp .6s
ease
forwards}.reveal:nth-child(2){animation-delay:.1s}.reveal:nth-child(3){animation-delay:.2s}.re
veal:nth-child(4){animation-delay:.3s}.reveal:nth-child(5){animation-delay:.4s}.reveal:nth-chil
d(6){animation-delay:.5s}@keyframes
fadeUp{to{opacity:1;transform:translateY(0)}}.metric-card{transition:transform .2s
ease,border-color .2s
ease}.metric-card:hover{transform:translateY(-2px);border-color:var(--accent)}.tbl-row{transiti
on:background .15s
ease}.tbl-row:hover{background:rgba(196,112,70,.06)}.tab-btn{transition:all .15s
ease}.tab-btn.active{background:var(--accent-soft);border-bottom:2px solid
var(--accent);color:var(--text)}.arch-node{transition:all .2s
ease}.arch-node:hover{filter:brightness(1.15)}.pulse-dot{animation:pulse 2s
infinite}@keyframes pulse{0%,100%{opacity:1}50%{opacity:.4}}.flow-line{stroke-dasharray:6
4;animation:dash 1.5s linear infinite}@keyframes dash{to{stroke-dashoffset:-20}}';
  document.head.appendChild(s);
})();
</script>
<div style="background-color:transparent;box-sizing:border-box;">
  <div style="color:var(--text);font-family:'Sora',sans-serif;background:radial-gradient(1200px
600px at 80% -10%, rgba(196,112,70,.08), transparent 60%), var(--bg);min-height:100%;">
    <div class="max-w-6xl mx-auto px-6 py-12">
```

```
    <!-- HERO -->
    <div class="reveal mb-14">
      <div style="display:flex;align-items:center;gap:14px;margin-bottom:16px;">
        <div
style="width:44px;height:44px;border-radius:12px;background:linear-gradient(135deg,var(--accent),var(--accent-soft));display:flex;align-items:center;justify-content:center;color:var(--text);flex-shrink:0;">
          <svg width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2"><rect x="3" y="3" width="7" height="7" rx="1"/><rect x="14" y="3" width="7"
height="7" rx="1"/><rect x="14" y="14" width="7" height="7" rx="1"/><rect x="3" y="14"
width="7" height="7" rx="1"/></svg>
        </div>
        <div>
          <p
style="font-family:monospace;font-size:11px;letter-spacing:0.2em;text-transform:uppercase;color:var(--muted)">Production Architecture Blueprint</p>
          <h1 class="text-4xl font-bold"
style="color:var(--text);letter-spacing:-0.02em;line-height:1.15">
            Gemma AI Client · <span
style="background:linear-gradient(100deg,var(--accent),var(--accent2));-webkit-background-clip:text;background-clip:text;color:transparent">Dark Mistral</span>
          </h1>
        </div>
      </div>
      <p style="color:var(--muted);max-width:680px;line-height:1.7" class="text-base mt-3">
        From prototype shell to production-grade AI workstation. Complete technical gap
analysis, layered system architecture, implementation phases, and priority roadmap.
      </p>
    </div>

    <!-- METRIC CARDS -->
    <div class="reveal flex flex-wrap gap-4 mb-14">
      <div class="metric-card rounded-2xl p-5" style="flex:1 1
180px;background:var(--card);border:1px solid var(--border);">
        <div style="display:flex;align-items:center;gap:8px;margin-bottom:8px;">
          <div
style="width:32px;height:32px;border-radius:8px;background:var(--accent-soft);display:flex;align-items:center;justify-content:center;color:var(--accent);">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2"><path d="M12 2L2 7l10 5 10-5-10-5z"/><path d="M2 17l10 5 10-5"/><path
d="M2 12l10 5 10-5"/></svg>
          </div>
          <div style="color:var(--muted)" class="text-xs uppercase tracking-wide
font-medium">Architecture Layers</div>
        </div>
        <div style="color:var(--text)" class="text-3xl font-bold">5</div>
        <div class="text-xs mt-1" style="color:var(--muted)">UI → Hooks → Domain →
Services → Infra</div>
```

```
    </div>
    <div class="metric-card rounded-2xl p-5" style="flex:1 1
180px;background:var(--card);border:1px solid var(--border);">
      <div style="display:flex;align-items:center;gap:8px;margin-bottom:8px;">
        <div
style="width:32px;height:32px;border-radius:8px;background:var(--accent-soft);display:flex;al
ign-items:center;justify-content:center;color:var(--accent);">
          <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2"><circle cx="12" cy="12" r="10"/><path d="M12 6v6l4 2"/></svg>
        </div>
        <div style="color:var(--muted)" class="text-xs uppercase tracking-wide
font-medium">Implementation Phases</div>
      </div>
      <div style="color:var(--text)" class="text-3xl font-bold">6</div>
      <div class="text-xs mt-1" style="color:var(--muted)">UI → Connectivity → Multimodal
→ Persistence → Agents → Platform</div>
    </div>
    <div class="metric-card rounded-2xl p-5" style="flex:1 1
180px;background:var(--card);border:1px solid var(--border);">
      <div style="display:flex;align-items:center;gap:8px;margin-bottom:8px;">
        <div
style="width:32px;height:32px;border-radius:8px;background:rgba(220,74,56,.14);display:fle
x;align-items:center;justify-content:center;color:#dc4a38;">
          <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2"><path d="M12 9v4"/><path d="M12 17h.01"/><path d="M10.29 3.86L1.82
18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"/></svg>
        </div>
        <div style="color:var(--muted)" class="text-xs uppercase tracking-wide
font-medium">P0 Critical Gaps</div>
      </div>
      <div style="color:var(--text)" class="text-3xl font-bold">7</div>
      <div class="text-xs mt-1" style="color:var(--muted)">API, Security, Streaming, Errors,
Cancel, Persistence, Schema</div>
    </div>
    <div class="metric-card rounded-2xl p-5" style="flex:1 1
180px;background:var(--card);border:1px solid var(--border);">
      <div style="display:flex;align-items:center;gap:8px;margin-bottom:8px;">
        <div
style="width:32px;height:32px;border-radius:8px;background:var(--accent2-soft);display:flex;
align-items:center;justify-content:center;color:var(--accent2);">
          <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0
2-2V8z"/><polyline points="14 2 14 8 20 8"/></svg>
        </div>
        <div style="color:var(--muted)" class="text-xs uppercase tracking-wide
font-medium">Technical Gaps Identified</div>
      </div>
      <div style="color:var(--text)" class="text-3xl font-bold">65</div>
```

```
        <div class="text-xs mt-1" style="color:var(--muted)">Architecture, features, infrastructure, and UX details</div>
      </div>
    </div>

    <!-- ARCHITECTURE DIAGRAM -->
    <div class="reveal rounded-2xl p-8 mb-8" style="background:var(--card);border:1px solid var(--border);">
      <div style="display:flex;align-items:center;justify-content:space-between;margin-bottom:24px;flex-wrap:wrap;gap:12px;">
        <div>
          <h2 class="text-xl font-semibold" style="color:var(--text)">Target System Architecture</h2>
          <p class="text-sm mt-1" style="color:var(--muted)">Five cleanly separated layers — UI never owns API credentials</p>
        </div>
        <div style="display:flex;gap:8px;flex-wrap:wrap;">
          <span style="padding:4px 10px;border-radius:20px;background:var(--accent-soft);color:var(--text);border:1px solid var(--accent);font-size:11px;font-weight:600;">Frontend</span>
          <span style="padding:4px 10px;border-radius:20px;background:var(--accent2-soft);color:var(--text);border:1px solid var(--accent2);font-size:11px;font-weight:600;">API Gateway</span>
          <span style="padding:4px 10px;border-radius:20px;background:rgba(123,168,184,.14);color:var(--text);border:1px solid rgba(123,168,184,.3);font-size:11px;font-weight:600;">Infrastructure</span>
        </div>
      </div>

      <svg viewBox="0 0 800 520" style="width:100%;height:auto;">
        <!-- Layer 1: User -->
        <g class="arch-node">
          <rect x="320" y="10" width="160" height="44" rx="8" fill="var(--accent-soft)" stroke="var(--accent)" stroke-width="1.5"/>
          <text x="400" y="37" text-anchor="middle" fill="var(--text)" font-size="13" font-weight="600" font-family="Sora,sans-serif">USER</text>
        </g>

        <!-- Flow line -->
        <line x1="400" y1="54" x2="400" y2="74" stroke="var(--muted)" stroke-width="1.5" class="flow-line"/>

        <!-- Layer 2: React/Vite UI -->
        <g class="arch-node">
          <rect x="180" y="74" width="440" height="60" rx="10" fill="var(--bg)" stroke="var(--accent)" stroke-width="1.5"/>
```

```
        <text x="400" y="98" text-anchor="middle" fill="var(--text)" font-size="12"
font-weight="600" font-family="Sora,sans-serif">React / Vite · Mistral-style UI</text>
        <text x="400" y="118" text-anchor="middle" fill="var(--muted)" font-size="10"
font-family="Sora,sans-serif">Sidebar · Conversation · Composer · Settings · Model
Selector</text>
      </g>

      <line x1="400" y1="134" x2="400" y2="154" stroke="var(--muted)" stroke-width="1.5"
class="flow-line"/>
      <text x="408" y="148" fill="var(--muted)" font-size="9"
font-family="monospace">HTTPS / SSE</text>

      <!-- Layer 3: API Gateway -->
      <g class="arch-node">
        <rect x="140" y="154" width="520" height="60" rx="10" fill="var(--bg)"
stroke="var(--accent2)" stroke-width="1.5"/>
        <text x="400" y="178" text-anchor="middle" fill="var(--text)" font-size="12"
font-weight="600" font-family="Sora,sans-serif">API Gateway</text>
        <text x="400" y="198" text-anchor="middle" fill="var(--muted)" font-size="10"
font-family="Sora,sans-serif">Auth · Validation · Rate Limiting · Streaming
Orchestration</text>
      </g>

      <line x1="400" y1="214" x2="400" y2="234" stroke="var(--muted)" stroke-width="1.5"
class="flow-line"/>

      <!-- Layer 4: Services Row -->
      <g class="arch-node">
        <rect x="60" y="234" width="200" height="56" rx="10" fill="var(--bg)"
stroke="var(--border)" stroke-width="1.5"/>
        <text x="160" y="258" text-anchor="middle" fill="var(--text)" font-size="11"
font-weight="600" font-family="Sora,sans-serif">Chat Service</text>
        <text x="160" y="276" text-anchor="middle" fill="var(--muted)" font-size="9"
font-family="Sora,sans-serif">streamChat · cancel · retry</text>
      </g>
      <g class="arch-node">
        <rect x="300" y="234" width="200" height="56" rx="10" fill="var(--bg)"
stroke="var(--border)" stroke-width="1.5"/>
        <text x="400" y="258" text-anchor="middle" fill="var(--text)" font-size="11"
font-weight="600" font-family="Sora,sans-serif">File Service</text>
        <text x="400" y="276" text-anchor="middle" fill="var(--muted)" font-size="9"
font-family="Sora,sans-serif">upload · validate · store</text>
      </g>
      <g class="arch-node">
        <rect x="540" y="234" width="200" height="56" rx="10" fill="var(--bg)"
stroke="var(--border)" stroke-width="1.5"/>
        <text x="640" y="258" text-anchor="middle" fill="var(--text)" font-size="11"
font-weight="600" font-family="Sora,sans-serif">Tool Service</text>
```

<text x="640" y="276" text-anchor="middle" fill="var(--muted)" font-size="9" font-family="Sora,sans-serif">function calling · search</text>
</g>

<line x1="160" y1="290" x2="160" y2="310" stroke="var(--muted)" stroke-width="1.5" class="flow-line"/>
<line x1="400" y1="290" x2="400" y2="310" stroke="var(--muted)" stroke-width="1.5" class="flow-line"/>
<line x1="640" y1="290" x2="640" y2="310" stroke="var(--muted)" stroke-width="1.5" class="flow-line"/>

<!-- Layer 5: AI Provider + Storage -->
<g class="arch-node">
<rect x="40" y="310" width="240" height="60" rx="10" fill="var(--bg)" stroke="var(--accent)" stroke-width="1.5"/>
<text x="160" y="334" text-anchor="middle" fill="var(--text)" font-size="11" font-weight="600" font-family="Sora,sans-serif">AI Provider Abstraction</text>
<text x="160" y="354" text-anchor="middle" fill="var(--muted)" font-size="9" font-family="Sora,sans-serif">Google/Gemma · Local Gemma · Other AI</text>
</g>
<g class="arch-node">
<rect x="320" y="310" width="160" height="60" rx="10" fill="var(--bg)" stroke="var(--accent2)" stroke-width="1.5"/>
<text x="400" y="334" text-anchor="middle" fill="var(--text)" font-size="11" font-weight="600" font-family="Sora,sans-serif">Object Store</text>
<text x="400" y="354" text-anchor="middle" fill="var(--muted)" font-size="9" font-family="Sora,sans-serif">images · PDFs · files</text>
</g>
<g class="arch-node">
<rect x="520" y="310" width="240" height="60" rx="10" fill="var(--bg)" stroke="var(--accent2)" stroke-width="1.5"/>
<text x="640" y="334" text-anchor="middle" fill="var(--text)" font-size="11" font-weight="600" font-family="Sora,sans-serif">Functions / Tools</text>
<text x="640" y="354" text-anchor="middle" fill="var(--muted)" font-size="9" font-family="Sora,sans-serif">custom functions · Google Search</text>
</g>

<line x1="160" y1="370" x2="160" y2="390" stroke="var(--muted)" stroke-width="1.5" class="flow-line"/>

<!-- Layer 6: Models -->
<g class="arch-node">
<rect x="60" y="390" width="100" height="50" rx="8" fill="var(--accent-soft)" stroke="var(--accent)" stroke-width="1"/>
<text x="110" y="412" text-anchor="middle" fill="var(--text)" font-size="10" font-weight="600" font-family="Sora,sans-serif">Gemma</text>
<circle cx="110" cy="428" r="3" fill="var(--accent)" class="pulse-dot"/>
</g>

```
        <g class="arch-node">
          <rect x="180" y="390" width="100" height="50" rx="8" fill="var(--accent-soft)"
stroke="var(--accent)" stroke-width="1"/>
          <text x="230" y="412" text-anchor="middle" fill="var(--text)" font-size="10"
font-weight="600" font-family="Sora,sans-serif">Gemini</text>
          <circle cx="230" cy="428" r="3" fill="var(--accent)" class="pulse-dot"/>
        </g>
        <g class="arch-node">
          <rect x="300" y="390" width="100" height="50" rx="8" fill="var(--accent2-soft)"
stroke="var(--accent2)" stroke-width="1"/>
          <text x="350" y="412" text-anchor="middle" fill="var(--text)" font-size="10"
font-weight="600" font-family="Sora,sans-serif">Local</text>
          <text x="350" y="428" text-anchor="middle" fill="var(--muted)" font-size="8"
font-family="Sora,sans-serif">Ollama / llama.cpp</text>
        </g>

        <!-- Database -->
        <g class="arch-node" transform="translate(500, 390)">
          <ellipse cx="70" cy="10" rx="60" ry="8" fill="none" stroke="var(--accent2)"
stroke-width="1.5"/>
          <path d="M10 10 L10 40 Q10 48 70 48 Q130 48 130 40 L130 10" fill="var(--bg)"
stroke="var(--accent2)" stroke-width="1.5"/>
          <ellipse cx="70" cy="25" rx="60" ry="8" fill="none" stroke="var(--accent2)"
stroke-width="1" stroke-dasharray="3 3"/>
          <text x="70" y="35" text-anchor="middle" fill="var(--text)" font-size="10"
font-weight="600" font-family="Sora,sans-serif">PostgreSQL</text>
        </g>

        <!-- Key decision callout -->
        <g transform="translate(40, 460)">
          <rect x="0" y="0" width="720" height="48" rx="8" fill="rgba(196,112,70,.08)"
stroke="var(--accent)" stroke-width="1" stroke-dasharray="4 3"/>
          <text x="16" y="22" fill="var(--accent)" font-size="11" font-weight="700"
font-family="Sora,sans-serif">KEY DECISION</text>
          <text x="16" y="40" fill="var(--muted)" font-size="10"
font-family="Sora,sans-serif">Introduce AIProvider interface immediately — prevents the
entire app from becoming tightly coupled to one provider. Chat UI → ChatService →
AIProvider → (Google / Local / Other)</text>
        </g>
      </svg>
    </div>

    <!-- PRIORITY MATRIX -->
    <div class="reveal rounded-2xl p-8 mb-8" style="background:var(--card);border:1px solid
var(--border);">
      <h2 class="text-xl font-semibold mb-2" style="color:var(--text)">Priority Matrix</h2>
      <p class="text-sm mb-6" style="color:var(--muted)">Critical gaps sorted by
implementation urgency</p>
```

<table class="w-full text-sm" style="border-collapse:collapse;">
<thead>
<tr style="color:var(--muted);border-bottom:1px solid var(--border)" class="text-xs uppercase tracking-wide">
<th class="text-left pb-3 font-medium" style="width:80px;">Priority</th>
<th class="text-left pb-3 font-medium">Missing Component</th>
<th class="text-left pb-3 font-medium" style="width:120px;">Importance</th>
<th class="text-left pb-3 font-medium" style="width:120px;">Phase</th>
</tr>
</thead>
<tbody style="color:var(--text)">
<tr class="tbl-row" style="border-bottom:1px solid var(--border)">
<td class="py-3.5">P0</td>
<td class="py-3.5 font-medium">Real API / Backend boundary · API key security</td>
<td class="py-3.5" style="color:#f87171;font-weight:600;">Critical</td>
<td class="py-3.5" style="color:var(--muted);font-size:12px;">Phase 2</td>
</tr>
<tr class="tbl-row" style="border-bottom:1px solid var(--border)">
<td class="py-3.5">P0</td>
<td class="py-3.5 font-medium">Streaming responses · AbortController cancellation</td>
<td class="py-3.5" style="color:#f87171;font-weight:600;">Critical</td>
<td class="py-3.5" style="color:var(--muted);font-size:12px;">Phase 2</td>
</tr>
<tr class="tbl-row" style="border-bottom:1px solid var(--border)">
<td class="py-3.5">P0</td>
<td class="py-3.5 font-medium">Error handling · Retry logic · Granular status states</td>
<td class="py-3.5" style="color:#f87171;font-weight:600;">Critical</td>
<td class="py-3.5" style="color:var(--muted);font-size:12px;">Phase 1</td>
</tr>
<tr class="tbl-row" style="border-bottom:1px solid var(--border)">
<td class="py-3.5">P0</td>
<td class="py-3.5 font-medium">Persistent conversation state · Proper message schema</td>
<td class="py-3.5" style="color:#f87171;font-weight:600;">Critical</td>
<td class="py-3.5" style="color:var(--muted);font-size:12px;">Phase 1</td>
</tr>

<tr class="tbl-row" style="border-bottom:1px solid var(--border)">
<td class="py-3.5">P1</td>
<td class="py-3.5 font-medium">Markdown renderer · Code highlighting · Copy buttons</td>
<td class="py-3.5" style="color:#fb923c;font-weight:600;">High</td>
<td class="py-3.5" style="color:var(--muted);font-size:12px;">Phase 1</td>
</tr>
<tr class="tbl-row" style="border-bottom:1px solid var(--border)">
<td class="py-3.5">P1</td>
<td class="py-3.5 font-medium">Conversation sidebar · Model selector · Multiline composer</td>
<td class="py-3.5" style="color:#fb923c;font-weight:600;">High</td>
<td class="py-3.5" style="color:var(--muted);font-size:12px;">Phase 1</td>
</tr>
<tr class="tbl-row" style="border-bottom:1px solid var(--border)">
<td class="py-3.5">P1</td>
<td class="py-3.5 font-medium">Attachments · Retry/regenerate · Auto-scroll · Settings</td>
<td class="py-3.5" style="color:#fb923c;font-weight:600;">High</td>
<td class="py-3.5" style="color:var(--muted);font-size:12px;">Phase 1</td>
</tr>
<tr class="tbl-row" style="border-bottom:1px solid var(--border)">
<td class="py-3.5">P2</td>
<td class="py-3.5 font-medium">Authentication · Database · Search · Export/Import</td>
<td class="py-3.5" style="color:#facc15;font-weight:600;">Production</td>
<td class="py-3.5" style="color:var(--muted);font-size:12px;">Phase 4</td>
</tr>
<tr class="tbl-row" style="border-bottom:1px solid var(--border)">
<td class="py-3.5">P2</td>
<td class="py-3.5 font-medium">Tool calling · Search grounding · Structured output · RAG</td>
<td class="py-3.5" style="color:#facc15;font-weight:600;">Advanced</td>
<td class="py-3.5" style="color:var(--muted);font-size:12px;">Phase 5</td>
</tr>
<tr class="tbl-row">

```
          <td class="py-3.5"><span style="padding:3px
10px;border-radius:20px;background:rgba(123,168,184,.14);color:var(--accent2);border:1px
solid rgba(123,168,184,.3);font-size:11px;font-weight:700;">P3</span></td>
          <td class="py-3.5 font-medium">Conversation branching · Local inference ·
PWA/offline · Multi-provider</td>
          <td class="py-3.5" style="color:var(--accent2);font-weight:600;">Strategic</td>
          <td class="py-3.5" style="color:var(--muted);font-size:12px;">Phase 6</td>
         </tr>
        </tbody>
       </table>
      </div>

      <!-- PHASE ROADMAP -->
      <div class="reveal rounded-2xl p-8 mb-8" style="background:var(--card);border:1px solid
var(--border);">
       <div
style="display:flex;align-items:center;justify-content:space-between;margin-bottom:20px;flex-
wrap:wrap;gap:12px;">
        <div>
         <h2 class="text-xl font-semibold" style="color:var(--text)">Implementation
Roadmap</h2>
         <p class="text-sm mt-1" style="color:var(--muted)">Six phases from prototype to AI
workstation</p>
        </div>
       </div>

       <!-- Phase Tabs -->
       <div style="display:flex;gap:4px;flex-wrap:wrap;margin-bottom:20px;border-bottom:1px
solid var(--border);">
        <button class="tab-btn active" onclick="showPhase(0, this)" style="padding:10px
16px;background:transparent;border:none;border-bottom:2px solid
transparent;color:var(--muted);cursor:pointer;font-size:13px;font-weight:600;font-family:Sora,
sans-serif;">Phase 1 · Real Chat UI</button>
        <button class="tab-btn" onclick="showPhase(1, this)" style="padding:10px
16px;background:transparent;border:none;border-bottom:2px solid
transparent;color:var(--muted);cursor:pointer;font-size:13px;font-weight:600;font-family:Sora,
sans-serif;">Phase 2 · Connectivity</button>
        <button class="tab-btn" onclick="showPhase(2, this)" style="padding:10px
16px;background:transparent;border:none;border-bottom:2px solid
transparent;color:var(--muted);cursor:pointer;font-size:13px;font-weight:600;font-family:Sora,
sans-serif;">Phase 3 · Multimodal</button>
        <button class="tab-btn" onclick="showPhase(3, this)" style="padding:10px
16px;background:transparent;border:none;border-bottom:2px solid
transparent;color:var(--muted);cursor:pointer;font-size:13px;font-weight:600;font-family:Sora,
sans-serif;">Phase 4 · Persistence</button>
        <button class="tab-btn" onclick="showPhase(4, this)" style="padding:10px
16px;background:transparent;border:none;border-bottom:2px solid
```

```
transparent;color:var(--muted);cursor:pointer;font-size:13px;font-weight:600;font-family:Sora, sans-serif;">Phase 5 · Agents</button>
        <button class="tab-btn" onclick="showPhase(5, this)" style="padding:10px 16px;background:transparent;border:none;border-bottom:2px solid transparent;color:var(--muted);cursor:pointer;font-size:13px;font-weight:600;font-family:Sora, sans-serif;">Phase 6 · Platform</button>
      </div>

      <!-- Phase Content -->
      <div id="phase-content" style="min-height:240px;">
        <div id="phase-0">
          <div style="display:flex;align-items:center;gap:10px;margin-bottom:16px;">
            <span style="width:32px;height:32px;border-radius:8px;background:var(--accent-soft);display:flex;align-items:center;justify-content:center;color:var(--accent);font-weight:700;font-size:14px;">1</span>
            <h3 style="color:var(--text);font-size:16px;font-weight:700;margin:0;">Turn prototype into a real chat UI</h3>
          </div>
          <div style="display:grid;grid-template-columns:repeat(auto-fill,minmax(220px,1fr));gap:12px;">
            <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid var(--border);">
              <div style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Multiline Composer</div>
              <div style="color:var(--muted);font-size:11px;line-height:1.5;">textarea auto-grow · Enter=send · Shift+Enter=newline</div>
            </div>
            <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid var(--border);">
              <div style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Proper Message Model</div>
              <div style="color:var(--muted);font-size:11px;line-height:1.5;">status enum · usage tokens · multimodal content parts</div>
            </div>
            <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid var(--border);">
              <div style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Markdown + Code</div>
              <div style="color:var(--muted);font-size:11px;line-height:1.5;">sanitized renderer · syntax highlighting · copy buttons</div>
            </div>
            <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid var(--border);">
```

<div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Message
Actions
copy · retry ·
regenerate · stop generation

<div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
<div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Auto-scroll
intelligent: only scroll
when user is at bottom

<div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
<div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Sidebar +
Persistence
new conversation ·
localStorage · grouped by date

<div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
<div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Error
States
retry UI · distinct
error codes · failed message rendering

<div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
<div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Model
Selector
capability-driven UI ·
from config/API

<span
style="width:32px;height:32px;border-radius:8px;background:var(--accent-soft);display:flex;al
ign-items:center;justify-content:center;color:var(--accent);font-weight:700;font-size:14px;">2<
/span>
<h3 style="color:var(--text);font-size:16px;font-weight:700;margin:0;">Real Gemma
connectivity via @google/genai</h3>

```
        </div>
        <div
style="display:grid;grid-template-columns:repeat(auto-fill,minmax(220px,1fr));gap:12px;">
          <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
            <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Backend API
Layer</div>
            <div style="color:var(--muted);font-size:11px;line-height:1.5;">Node server ·
credentials never exposed to browser</div>
          </div>
          <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
            <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Google GenAI
SDK</div>
            <div style="color:var(--muted);font-size:11px;line-height:1.5;">@google/genai ·
Interactions API for stateful flows</div>
          </div>
          <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
            <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">SSE
Streaming</div>
            <div style="color:var(--muted);font-size:11px;line-height:1.5;">token-by-token ·
AsyncIterable · event parsing</div>
          </div>
          <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
            <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">AbortController</
div>
            <div style="color:var(--muted);font-size:11px;line-height:1.5;">Stop button · cancel
signal · status: "cancelled"</div>
          </div>
          <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
            <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Error
Translation</div>
            <div style="color:var(--muted);font-size:11px;line-height:1.5;">API errors →
structured ApiError · retryable flag</div>
          </div>
          <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
            <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Token
Usage</div>
```

input/output tokens · context budget tracking

Generation Params
temperature · topP · topK · maxOutputTokens

Retries with Backoff
exponential backoff · preserve partial responses

3
<h3 style="color:var(--text);font-size:16px;font-weight:700;margin:0;">Multimodal input support</h3>

Image Input
upload · paste · drag & drop · preview thumbnails

PDF Documents
file upload · MIME validation · size limits

```
          </div>
          <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
            <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Audio Input</div>
            <div style="color:var(--muted);font-size:11px;line-height:1.5;">voice messages ·
where model supports it</div>
          </div>
          <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
            <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Video Input</div>
            <div style="color:var(--muted);font-size:11px;line-height:1.5;">frame extraction ·
capability-gated UI</div>
          </div>
          <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
            <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Attachment
Model</div>
            <div style="color:var(--muted);font-size:11px;line-height:1.5;">status:
uploading/processing/ready/error</div>
          </div>
          <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
            <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Model Capability
Registry</div>
            <div style="color:var(--muted);font-size:11px;line-height:1.5;">dynamic UI based
on what selected model supports</div>
          </div>
        </div>
      </div>

      <div id="phase-3" style="display:none;">
        <div style="display:flex;align-items:center;gap:10px;margin-bottom:16px;">
          <span
style="width:32px;height:32px;border-radius:8px;background:var(--accent-soft);display:flex;al
ign-items:center;justify-content:center;color:var(--accent);font-weight:700;font-size:14px;">4<
/span>
          <h3 style="color:var(--text);font-size:16px;font-weight:700;margin:0;">Persistence,
authentication & search</h3>
        </div>
        <div
style="display:grid;grid-template-columns:repeat(auto-fill,minmax(220px,1fr));gap:12px;">
          <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
```

<div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Authentication</d
iv>
OAuth / Auth
provider · sessions · user context

<div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
<div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">PostgreSQL
Storage
conversations ·
messages · users · settings

<div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
<div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Conversation
Search
Ctrl+K · search titles
& content · shortcuts

<div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
<div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Auto Titles
LLM-generated
conversation titles from first message

<div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
<div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Rename /
Archive / Delete
full conversation
management · confirmations

<div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
<div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Export /
Import
Markdown · JSON ·
TXT · versioned schema

```
        <div id="phase-4" style="display:none;">
          <div style="display:flex;align-items:center;gap:10px;margin-bottom:16px;">
            <span
style="width:32px;height:32px;border-radius:8px;background:var(--accent-soft);display:flex;al
ign-items:center;justify-content:center;color:var(--accent);font-weight:700;font-size:14px;">5<
/span>
            <h3 style="color:var(--text);font-size:16px;font-weight:700;margin:0;">Agent
capabilities & tool use</h3>
          </div>
          <div
style="display:grid;grid-template-columns:repeat(auto-fill,minmax(220px,1fr));gap:12px;">
            <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
              <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Function
Calling</div>
              <div style="color:var(--muted);font-size:11px;line-height:1.5;">custom tool
definitions · schema validation</div>
            </div>
            <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
              <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Google
Search</div>
              <div style="color:var(--muted);font-size:11px;line-height:1.5;">built-in managed tool
· grounding with sources</div>
            </div>
            <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
              <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Structured
Output</div>
              <div style="color:var(--muted);font-size:11px;line-height:1.5;">JSON schema ·
validated objects · component rendering</div>
            </div>
            <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
              <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Tool Execution
UI</div>
              <div style="color:var(--muted);font-size:11px;line-height:1.5;">tool call parts ·
status indicators · result display</div>
            </div>
            <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
              <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Message Part
Model</div>
```

TextPart · ToolCallPart · ToolResultPart · FilePart

Interactions API
Google's recommended primitive for stateful agent flows

6
<h3 style="color:var(--text);font-size:16px;font-weight:700;margin:0;">Advanced platform — Gemma Studio</h3>

Conversation Branching
parentMessageId · branch navigation · 1/3 selector

Local Inference
Ollama / llama.cpp · WebGPU/WASM exploration

Multi-Provider
OpenAI · Anthropic · Hugging Face · LM Studio

```
            </div>
            <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
              <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">RAG /
Knowledge</div>
              <div style="color:var(--muted);font-size:11px;line-height:1.5;">vector store ·
document indexing · retrieval</div>
            </div>
            <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
              <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">PWA /
Offline</div>
              <div style="color:var(--muted);font-size:11px;line-height:1.5;">service worker ·
cached shell · draft preservation</div>
            </div>
            <div style="padding:14px;border-radius:10px;background:var(--bg);border:1px solid
var(--border);">
              <div
style="color:var(--text);font-weight:600;font-size:13px;margin-bottom:4px;">Observability</di
v>
              <div style="color:var(--muted);font-size:11px;line-height:1.5;">request IDs · latency
· token metrics · error tracking</div>
            </div>
          </div>
        </div>
      </div>
    </div>

    <!-- FILE STRUCTURE -->
    <div class="reveal rounded-2xl p-8 mb-8" style="background:var(--card);border:1px solid
var(--border);">
      <h2 class="text-xl font-semibold mb-2" style="color:var(--text)">Expanded Project
Structure</h2>
      <p class="text-sm mb-6" style="color:var(--muted)">From prototype flat files to a
scalable architecture</p>

      <div
style="display:grid;grid-template-columns:repeat(auto-fill,minmax(280px,1fr));gap:20px;">
        <div>
          <div style="display:flex;align-items:center;gap:8px;margin-bottom:12px;">
            <div
style="width:6px;height:6px;border-radius:50%;background:var(--accent);"></div>
            <span style="color:var(--text);font-weight:700;font-size:13px;">Frontend
Application</span>
          </div>
```

<pre
style="font-family:monospace;font-size:11px;line-height:1.7;color:var(--muted);background:v
ar(--bg);padding:14px;border-radius:8px;border:1px solid
var(--border);overflow-x:auto;margin:0;">src/
├── app/
│ ├── App.tsx
│ ├── routes.tsx
│ └── providers.tsx
├── components/
│ ├── chat/
│ ├── composer/
│ ├── layout/
│ ├── model/
│ ├── settings/
│ └── ui/
├── hooks/
├── services/
│ ├── api/
│ ├── storage/
│ └── markdown/
├── stores/
├── types/
└── utils/</pre>

<div
style="width:6px;height:6px;border-radius:50%;background:var(--accent2);">
Backend API
Server

<pre
style="font-family:monospace;font-size:11px;line-height:1.7;color:var(--muted);background:v
ar(--bg);padding:14px;border-radius:8px;border:1px solid
var(--border);overflow-x:auto;margin:0;">server/
├── index.ts
├── routes/
│ ├── chat.ts
│ ├── models.ts
│ └── files.ts
├── services/
│ └── gemma.ts
└── middleware/
├── auth.ts
├── rateLimit.ts
└── errors.ts</pre>

```
        <div>
          <div style="display:flex;align-items:center;gap:8px;margin-bottom:12px;">
            <div style="width:6px;height:6px;border-radius:50%;background:#94a3b8;"></div>
            <span style="color:var(--text);font-weight:700;font-size:13px;">Infrastructure &
Config</span>
          </div>
          <pre
style="font-family:monospace;font-size:11px;line-height:1.7;color:var(--muted);background:v
ar(--bg);padding:14px;border-radius:8px;border:1px solid
var(--border);overflow-x:auto;margin:0;">tests/
├── components/
├── hooks/
└── services/

.env.example
package.json
vite.config.ts
tailwind.config.js
tsconfig.json
README.md</pre>
        </div>
      </div>
    </div>

    <!-- IMMEDIATE NEXT STEP -->
    <div class="reveal rounded-2xl p-8 mb-8"
style="background:linear-gradient(135deg,var(--accent-soft),transparent);border:1px solid
var(--accent);">
      <div style="display:flex;align-items:flex-start;gap:16px;flex-wrap:wrap;">
        <div
style="width:48px;height:48px;border-radius:12px;background:var(--accent);display:flex;align
-items:center;justify-content:center;color:var(--bg);flex-shrink:0;">
          <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2"><path d="M5 12h14"/><path d="M12 5l7 7-7 7"/></svg>
        </div>
        <div style="flex:1;min-width:240px;">
          <h3 style="color:var(--text);font-size:18px;font-weight:700;margin:0 0 8px 0;">The
Next Milestone</h3>
          <p style="color:var(--muted);line-height:1.7;margin:0 0 12px 0;">
            Do not simply "connect gemmaApi.ts." The correct next step is:
          </p>
          <div style="display:inline-block;padding:10px
18px;border-radius:10px;background:var(--card);border:1px solid
var(--border);color:var(--text);font-weight:600;font-size:14px;">
            Prototype → Real Streaming Chat Architecture
          </div>
          <p style="color:var(--muted);line-height:1.7;margin:12px 0 0 0;font-size:13px;">
```

Implement backend/API boundary, proper message/event types, streaming with cancellation, Markdown/code rendering, persistent conversations, error/retry handling, model selection, and a real composer first.

```
          </p>
        </div>
      </div>
    </div>

    <!-- Footer -->
    <div class="mt-16 pt-6" style="border-top:1px solid var(--border);">
      <p class="text-center text-xs" style="color:var(--muted)">Gemma AI Client Architecture · Dark Mistral Styling · Google GenAI SDK (@google/genai) · Interactions API ready</p>
    </div>
  </div>
 </div>
</div>

<script>
function showPhase(index, btn) {
 // Hide all phases
 for (var i = 0; i < 6; i++) {
  var el = document.getElementById('phase-' + i);
  if (el) el.style.display = 'none';
 }
 // Show selected
 var selected = document.getElementById('phase-' + index);
 if (selected) {
  selected.style.display = 'block';
  selected.style.animation = 'fadeUp .4s ease forwards';
 }
 // Update tabs
 var tabs = document.querySelectorAll('.tab-btn');
 tabs.forEach(function(t) { t.classList.remove('active'); });
 btn.classList.add('active');
}
</script>
</html>
```

Architecture Summary

The interactive blueprint above captures your complete gap analysis as a navigable system design. Key highlights:

Five-Layer Architecture — Clean separation from UI through to infrastructure, with the critical `AIProvider` abstraction preventing vendor lock-in. The browser never touches production API credentials.

Priority Matrix — 7 P0 critical gaps (real API, security, streaming, errors, cancellation, persistence, schema) form the foundation before any P1 features.

Six-Phase Roadmap — Click through the tabs to see each phase's deliverables:

1. Real Chat UI — Multiline composer, markdown, code highlighting, sidebar, persistence
2. Connectivity — Backend boundary, `@google/genai` SDK, SSE streaming, AbortController
3. Multimodal — Images, PDFs, audio, video with capability-gated UI
4. Persistence — Auth, PostgreSQL, search, export/import
5. Agents — Function calling, Google Search, structured output, tool execution UI
6. Platform — Branching, local inference, multi-provider, RAG, PWA

The critical architectural decision: Introduce the `AIProvider` interface immediately in `src/services/ai/` so your UI never knows it's talking to Google. This single decision is what lets you support Gemma, Gemini, local Ollama, and other providers without rebuilding the application.